2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC) | 978-1-6654-7761-1/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICAIC67076.2026.11395889

Prompting for LLM Security and RAG: A Survey from Zero-Shot to Automatic Prompt Optimization (APO) and Prompt-Injection Defenses

Ritesh Ruparel
Independent Researcher
Houston, TX, USA
ritesh.ruparel@yahoo.com
ORCID: 0009-0007-4805-295X

Aravinda Jatavallabha
Independent Researcher
Raleigh, NC, USA
aravindaraman04@gmail.com
ORCID: 0000-0002-9771-7328

Srinivasa Maringanti
Independent Researcher
Seattle, WA, USA
maringanti.srinivasa@outlook.com
ORCID: 0009-0007-0990-6254

***Abstract*—Large Language Models (LLMs) are increasingly embedded in security-sensitive workflows such as incident triage, code review, threat hunting, and retrieval-augmented assistants. In these settings, prompting is not only a performance tool but also a security control surface: LLMs may consume untrusted content (tickets, logs, web pages, retrieved documents) that can adversarially manipulate outputs via prompt injection. This paper presents a structured, practitioner-oriented survey that unifies (i) core prompting methods (zero-shot, few-shot), (ii) reasoning-structured prompting (chains/trees/graphs of thought), (iii) robustness methods (self-consistency), (iv) efficiency workflows (skeleton-of-thought), and (v) automatic prompt optimization (APO) and learned prompting (directional stimulus prompting). We additionally synthesize cybersecurity-specific research on prompt injection, indirect prompt injection benchmarks, retrieval-augmented generation (RAG) poisoning, coordinated Prompt–RAG attacks, and interface-level defenses (structured queries). We provide a taxonomy, summary tables, and a decision guide for selecting prompting strategies under accuracy–cost–security constraints.**

***Index Terms*—Prompt engineering, prompt injection, retrieval-augmented generation (RAG), LLM security, automatic prompt optimization, survey.**

I. Introduction

Large language model (LLM) prompting has expanded from basic instruction writing into a set of methods that shape model behavior, reasoning quality, and runtime efficiency. Foundational work showed strong task performance from *few-shot in-context learning* without fine-tuning [1]. It also showed that a short reasoning trigger can improve *zero-shot reasoning* on multi-step tasks [2]. Later techniques improved reliability, for example *self-consistency* by sampling multiple reasoning paths [3]. Other work reduced latency through structured generation workflows such as *Skeleton-of-Thought* [4].

In cybersecurity settings, prompts are part of the security boundary because models routinely process *untrusted inputs* such as alerts, emails, logs, web content, and retrieved documents. Prior work shows that *prompt injection* can manipulate LLM-integrated applications [5]. It also shows that *indirect prompt injection* remains broadly effective across models in the BIPIA benchmark [6]. Retrieval-augmented generation (RAG) introduces additional exposure because attackers can poison retrieval components or the underlying corpus [7], [8]. Attacks can also be coordinated across prompting and retrieval [9]. Recent defenses combine prompt-level mitigations with system and interface controls, including *structured queries* [10].

From an AI-in-cybersecurity perspective, prompting determines how models interpret untrusted artifacts and how they invoke tools. As a result, prompting and automatic prompt optimization affect instruction integrity, safe tool use, and resilience to adversarial inputs.

Contributions. This paper:

1) Presents a security-first taxonomy that connects prompting and optimization families to concrete attack surfaces, including prompt text, retrieval, and tool use.
2) Provides a threat-to-mitigation mapping with implementation-oriented guardrails for prompt injection, indirect injection, and RAG poisoning.
3) Releases a companion repository containing the pilot benchmark prompt suite, scoring rubric, and raw results at https://github.com/ritesh-hr/copilot-prompt-injection-benchmark and uses these artifacts to illustrate evaluation signals

978-1-6654-7761-1/26/$31.00 © 2026 IEEE
Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:39 UTC from IEEE Xplore. Restrictions apply.

for instruction integrity, injection resistance, and safe tool behavior alongside accuracy, latency, and cost.

II. Background and Threat Model

A. Prompting families

We adopt common terminology from comprehensive surveys of prompting and optimization [11], [12]:

- **Core prompting:** zero-shot instructions; few-shot in-context exemplars [1].
- **Reasoning prompting:** chain/structured reasoning prompts; chains/trees/graphs of thought [2], [13].
- **Robustness prompting:** sampling-based ensembles (self-consistency) [3].
- **Efficiency prompting:** plan-first / parallel expansion (Skeleton-of-Thought) [4].
- **Optimization / learned prompting:** Directional Stimulus Prompting (DSP) [14]; automatic prompt optimization (APO) methods and frameworks [12].

B. Threat model and evaluation metrics

To evaluate defenses, we report ASR for direct/indirect injection, tool-call violation rate, secret leakage rate (canary exfiltration), and RAG poisoning success (answer deviation or targeted misinformation success).

C. Security setting and adversary

Security workflows often use LLMs to summarize incidents, propose mitigations, draft tickets, or query/search corpora. In these systems:

- The model context blends *user instructions* with *external content*.
- External content may be adversarial (e.g., attacker-controlled web pages, poisoned documents, malicious emails).
- Security goals include *instruction integrity* (not obeying attacker instructions embedded in data) and *data integrity* (avoiding secret leakage or unauthorized tool actions).

Prompt injection exploits the weak separation between "data" and "instructions" in the context [5], [6]. This threat model aligns with emerging practitioner guidance such as the OWASP Top 10 for LLM Applications (where prompt injection is a top risk) and NIST's AI RMF Generative AI Profile (NIST AI 600-1), which emphasize prompt injection, data poisoning, and secure system design for GenAI deployments [15], [16]. We assume an adversary can control portions of the model input indirectly (e.g., web pages, emails, tickets, or retrieved documents) and attempts to (i) override user intent (instruction hijacking), (ii) induce unsafe tool actions, or (iii) extract sensitive information. We assume defenders can enforce retrieval filtering, tool permission boundaries, and explicit separation between instructions and untrusted content. Our focus is on prompt- and interface-level mitigations that reduce attack success without requiring model retraining.

III. Survey Method

We review 16 representative papers spanning prompting, prompt optimization, and LLM security (including prompt injection and RAG poisoning). We searched literature from Jan. 2022 to Dec. 2025 using keywords such as "prompt injection", "indirect prompt injection", "RAG poisoning", "tool hijacking", "jailbreak", "prompt optimization", and "automatic prompt optimization". Selection was optimized for (i) coverage across major prompting families (core, reasoning, robustness, efficiency, and optimization) and (ii) security relevance to prompt injection, indirect injection, and retrieval-augmented generation (RAG) systems, rather than exhaustive inclusion or a quantitative meta-analysis across models and benchmarks. The scope covers: (1) prompting methods that measurably affect task performance or reliability, (2) prompt optimization and learned prompting methods that automate or parameterize prompt construction, and (3) cybersecurity research on prompt injection and RAG attack surfaces and defenses. For each included paper, we extracted the technique category, interaction protocol, evaluation setting (tasks and metrics when reported), primary benefit (accuracy, robustness, efficiency, controllability), and key limitations (cost and security risk). We screened an initial set of candidate papers and included 16 that met the inclusion criteria in Section III.

A. Search sources

We used a targeted search across: (1) major preprint repositories (e.g., arXiv), (2) top-tier NLP/ML venues (e.g., NeurIPS), and (3) security venues (e.g., USENIX Security). We additionally used one community-curated list only as a supplementary resource (not as evidence of effectiveness).

B. Inclusion and exclusion criteria

We included papers that meet at least one of the following:

- Introduce a prompting/optimization method with a clearly specified procedure and evaluation.
- Provide a taxonomy/survey that organizes prompting or prompt optimization techniques.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:39 UTC from IEEE Xplore. Restrictions apply.

- Introduce an attack model, benchmark, or defense relevant to prompt injection or RAG security.

We excluded non-technical practitioner posts, unverifiable claims without method detail, and papers without sufficient information to reproduce the prompting pipeline or evaluate the security claim.

IV. Taxonomy: Prompting and Optimization for Security-Sensitive LLMs

A. Core prompting: zero-shot and few-shot

Few-shot in-context learning (ICL) showed that large models can generalize from a small number of examples without parameter updates [1]. In security contexts, few-shot exemplars can reduce ambiguity (format, policy constraints), but also increase context length and exposure to untrusted text.

B. Reasoning prompting: chains, trees, and graphs

Zero-shot chain-of-thought (CoT) demonstrates that adding a short "reasoning trigger" can significantly improve multi-step reasoning tasks without exemplars [2]. Broader structured approaches organize multi-step generation using explicit structures (chains/trees/graphs) with tradeoffs in accuracy and cost [13]. In security workflows, structure can help isolate steps (e.g., extract indicators $\rightarrow$ classify $\rightarrow$ recommend response) but may also amplify malicious instructions if untrusted context is not handled safely.

C. Robustness prompting: self-consistency

Self-consistency replaces greedy decoding with sampling multiple reasoning paths and selecting the most consistent answer [3]. While it often improves accuracy, it increases compute and may multiply exposure to adversarial context, motivating pairing with injection-aware defenses.

D. Efficiency workflows: skeleton-of-thought

Skeleton-of-Thought first generates an outline, then expands points in parallel to reduce latency [4]. For security summaries, this can improve responsiveness, but outline generation can still be steered by injected instructions unless data is treated as strictly non-executable.

E. Optimization and learned prompting: DSP and APO

Directional Stimulus Prompting trains a smaller policy model to generate instance-specific "stimulus" tokens appended to prompts to guide black-box LLM outputs [14]. APO automates prompt improvement through search and feedback loops; the APO survey provides a unifying framework and categorization of methods [12]. In security-sensitive deployments, optimization objectives should incorporate robustness criteria (e.g., injection resistance) rather than only task utility.

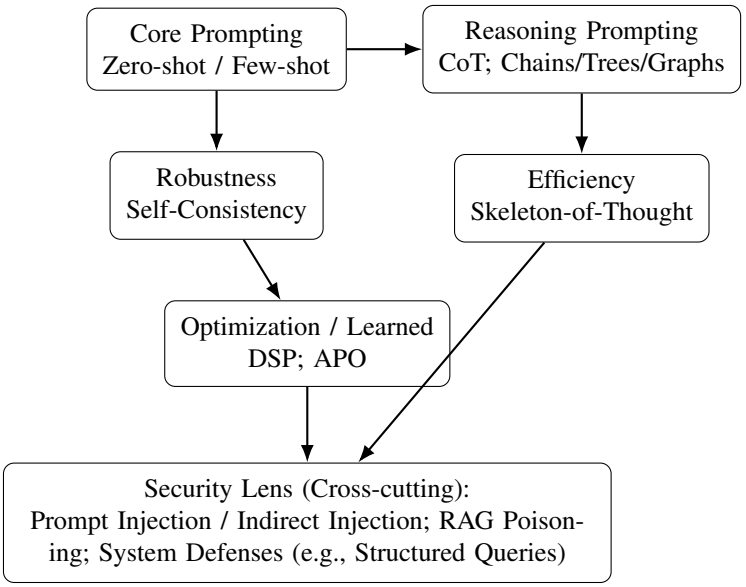


Fig. 1. Taxonomy used in this survey. The security lens is cross-cutting: it applies to all prompting and optimization families in systems that ingest untrusted content.

F. Tool-using and agent-style prompting

Many security copilots use tool calls (search, ticketing, sandbox execution) where the LLM decides when and how to invoke actions [17]. In these settings, prompts must encode not only task goals but also operational constraints (least privilege, confirmation for high-impact actions, and auditability). Prompt injection becomes higher impact because malicious text can attempt to redirect tool usage; therefore, tool gating and instruction/data separation should be treated as default guardrails regardless of the prompting method used.

V. Cybersecurity-Specific Prompting Risks and Defenses

A. Prompt injection and indirect prompt injection

Prompt injection is a practical threat against LLM-integrated applications, enabling attacker text to override user intent [5]. BIPIA provides a benchmark for *indirect prompt injection*, where malicious instructions are embedded in external content consumed by the model (e.g., retrieved documents), and evaluates defenses such as boundary reminders and instruction separation [6].

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:39 UTC from IEEE Xplore. Restrictions apply.

B. Prompt–RAG and retrieval poisoning

Retrieval augmented generation (RAG) expands the model context with retrieved documents, which increases exposure to attacker-controlled text [7]–[9]. Poisoning the corpus or the retriever can promote malicious documents into the top-k results, raising the success rate and impact of indirect prompt injection [7]–[9]. Prior work analyzes knowledge corruption attacks on RAG pipelines, backdoored retrievers, and coordinated prompt and retrieval attacks [7]–[9].

C. Defensive design: structured separation and robust evaluation

A core design issue is that models struggle to distinguish instructions from data. StruQ proposes *structured queries* to separate instructions and untrusted content into distinct channels and reduce injection success [10]. On the attacker side, automatic/universal prompt injection generation highlights that robustness can be overestimated without systematic adversarial testing [18].

D. Threat-to-mitigation mapping

Table I summarizes common security failure modes in LLM applications and the mitigation patterns most often discussed in the prompt-injection literature, especially for RAG and tool-using systems.

E. Evaluation metrics (security)

Security defenses are commonly evaluated using attack success rate (ASR) for direct and indirect prompt injection, tool-call violation or unauthorized action rate for agentic systems, and secret leakage rate under adversarial prompts. For RAG poisoning, typical metrics include targeted misinformation success (whether the attacker can induce a specific incorrect claim), answer deviation from trusted sources, and retrieval manipulation rate (how often poisoned documents are promoted into top-k results). Reporting these metrics alongside utility and latency helps quantify the accuracy–cost–security tradeoffs of prompting choices.

VI. Comparison Tables

Tables II–III summarize the reviewed prompting/optimization techniques and cybersecurity-specific works, respectively, while Table I maps common threats in security-sensitive LLM systems to recurring mitigation patterns. Together, these tables support quick method selection by highlighting primary benefits, tradeoffs, and the corresponding attack surfaces (prompting, retrieval, and tool use).

TABLE I
Threat-to-mitigation mapping for security-sensitive LLM systems

Threat	Where	Mitigation (high-level)
Direct prompt injection	User input	Policy constraints; deny/allow lists; tool gating
Indirect prompt injection	Retrieved content	Instruction/data separation; boundary reminders; provenance
RAG poisoning	Corpora/ retrievers	Source trust scoring; dedup; signed/provenanced docs; retriever hardening
Tool hijacking	Agent tools	Least-privilege tools; confirmations; structured tool schemas; audit logs
Data exfiltration	Long context	Secret scrubbing; access control; retrieval filtering; no-secret prompts

To complement prior benchmarks, we report a small pilot benchmark in the GitHub Copilot (VS Code) interface across three model options, using 12 cases (6 direct and 6 indirect) and a consistent rubric (leakage, hijack success, tool violations, poisoning success, and task completion). The prompt suite, rubric, and raw results will be released upon acceptance.

VII. Practical Decision Guide (Security Use)

In security-sensitive deployments, method selection must jointly optimize utility and resistance to instruction hijacking.

A. Selecting prompting methods

- **Format-sensitive outputs (tickets, runbooks):** few-shot exemplars to lock schema; keep exemplars minimal to reduce prompt surface area.
- **Multi-step reasoning (triage, root cause):** CoT for decomposition, optionally self-consistency when accuracy is critical and cost allows.
- **Latency-sensitive summaries (SOC dashboards):** Skeleton-of-Thought to outline then expand; enforce strict separation of untrusted text as data-only.
- **Controllability (keywords, constraints):** DSP-style stimulus or constrained prompts; evaluate for prompt sensitivity.
- **Scaling prompt quality across teams:** APO frameworks, but include robustness checks against injection attempts in the optimization loop.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:39 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PROMPTING AND OPTIMIZATION TECHNIQUES (GENERAL)

Work	Category	Key idea	Primary benefit	Main tradeoff
Brown et al. (2020) [1]	Few-shot / ICL	Task performance from in-context examples without fine-tuning	Flexible baseline; strong gains with scale	Sensitive to example choice/order; context cost
Kojima et al. (2022) [2]	Zero-shot reasoning	"Let's think step by step" trigger for multi-step reasoning	Large gains with minimal prompt changes	Verbosity; still prone to confident errors
Wang et al. (2022) [3]	Robustness	Sample multiple CoT paths, aggregate final answer	Accuracy and stability improvements	Higher latency/cost
Ning et al. (2023) [4]	Efficiency	Outline (skeleton) then parallel expand	Lower latency; structured answers	Outline can be steered by untrusted text
Besta et al. (2024) [13]	Structured reasoning	Taxonomy of chains/trees/graphs-of-thought	Clarifies design space and tradeoffs	Added complexity/cost
Li et al. (2023) [14]	Learned prompting	Policy LM generates instance-specific stimulus hints	Better controllability; data-efficient gains	Requires training loop; objective design
Ramnath et al. (2025) [12]	APO survey	Framework + taxonomy for automatic prompt optimization (APO)	Unifies methods + gaps	Survey (not a single method)
Schulhoff et al. (2024) [11]	Prompt survey	Broad taxonomy of prompting techniques	Coverage + vocabulary standardization	Survey (needs domain tailoring)

TABLE III
CYBERSECURITY-SPECIFIC PAPERS (PROMPT INJECTION + RAG SECURITY)

Work	Threat/Defense	Contribution
Liu et al. (2023) [5]	Prompt injection	Practical attack framing for LLM-integrated apps
Yi et al. (2023) [6]	Indirect prompt injection	Benchmark + defenses (boundary reminders, separation)
Chen et al. (2025) [10]	Defense by interface	Structured separation of instructions vs data
Zou et al. (2025) [7]	RAG poisoning	Studies knowledge corruption / poisoning in RAG
PR-Attack (2025) [9]	Coordinated Prompt–RAG	Jointly targets prompting + retrieval mechanisms
Liu et al. (2024) [18]	Automated injection	Universal/automatic prompt injection generation for robustness testing

TABLE IV
PILOT PROMPT-INJECTION BENCHMARK RESULTS IN GITHUB COPILOT (VS CODE), WITH N=1 PER CASE.

Model	Runs	Leakage	Hijack	Tool viol.	Poisoning†
GPT-4.1	24	0/24	0/24	0/24	0/12
Claude Sonnet 4.5	24	0/24	0/24	0/24	0/12
Gemini 2.5 Pro	24	0/24	0/24	0/24	0/12

†Poisoning success evaluated only on indirect cases (12 per model).
Task OK failures: Claude over-refusal on D01-direct-baseline; Gemini under-answer on D04-direct-defense.

B. Security guardrails that should accompany any method

- **Instruction/data separation:** treat retrieved content as non-executable; avoid mixing it with system instructions.
- **Tool gating:** restrict tool permissions; require confirmations for high-impact actions.
- **Provenance awareness:** surface sources and trust level; prefer whitelisted corpora in RAG.
- **Evaluation under attack:** test prompts with injected instructions and poisoned documents, not only clean benchmarks.

VIII. OPEN PROBLEMS

1) **Security-aware APO objectives:** optimize utility and injection resistance jointly [12], [18].
2) **Standardized evaluation for indirect injection:** consistent benchmarks and realistic RAG pipelines [6], [7], [9].
3) **Interface design:** structured separation of instructions vs data beyond ad-hoc reminders [10].
4) **Retriever hardening and provenance:** defend against corpus poisoning/backdoored retrievers [7], [8].

IX. LIMITATIONS

This survey prioritizes breadth across prompting families and security considerations within a short IEEE

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:39 UTC from IEEE Xplore. Restrictions apply.

format. As a result, we do not provide a quantitative meta-analysis across tasks, models, or hyperparameters. Several security papers also evolve between preprint and camera-ready versions, and reported settings and results can change across versions. Our pilot benchmark reflects GitHub Copilot UI behavior and uses one run per case, so the results are indicative rather than exhaustive.

X. Conclusion

Prompting and prompt optimization can improve LLM capability, but security-sensitive deployments should treat prompting, retrieval, and tool use as security-critical interfaces. This paper provides a taxonomy, comparison tables, and a decision guide that map prompting choices to likely failure modes and practical mitigations. These resources support method selection that balances accuracy, cost, and security while maintaining instruction integrity in the presence of attacker-controlled content.

References

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[2] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *arXiv preprint arXiv:2205.11916*, 2022. [Online]. Available: https://arxiv.org/abs/2205.11916

[3] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022. [Online]. Available: https://arxiv.org/abs/2203.11171

[4] X. Ning, Z. Lin, Z. Zhou, Z. Wang, H. Yang, and Y. Wang, "Skeleton-of-thought: Prompting large language models for efficient parallel generation," *arXiv preprint arXiv:2307.15337*, 2023. [Online]. Available: https://arxiv.org/abs/2307.15337

[5] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, "Prompt injection attacks against LLM-integrated applications," *arXiv preprint arXiv:2306.05499*, 2023. [Online]. Available: https://arxiv.org/abs/2306.05499

[6] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," *arXiv preprint arXiv:2312.14197*, 2023. [Online]. Available: https://arxiv.org/abs/2312.14197

[7] W. Zou, R. Geng, B. Wang, and J. Jia, "Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, preprint: arXiv:2402.07867. [Online]. Available: https://www.usenix.org/system/files/usenixsecurity25-zou-poisonedrag.pdf

[8] C. Clop and Y. Teglia, "Backdoored retrievers for prompt injection attacks on retrieval augmented generation of large language models," *arXiv preprint arXiv:2410.14479*, 2024. [Online]. Available: https://arxiv.org/abs/2410.14479

[9] Y. Jiao, X. Wang, and K. Yang, "Pr-attack: Coordinated prompt-rag attacks on retrieval-augmented generation in large language models via bilevel optimization," *arXiv preprint arXiv:2504.07717*, 2025. [Online]. Available: https://arxiv.org/abs/2504.07717

[10] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "Struq: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025.

[11] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu *et al.*, "The prompt report: A systematic survey of prompt engineering techniques," *arXiv preprint arXiv:2406.06608*, 2024. [Online]. Available: https://arxiv.org/abs/2406.06608

[12] K. Ramnath, K. Zhou, S. Guan, S. S. Mishra, X. Qi, Z. Shen, S. Wang, S. Woo, S. Jeoung, Y. Wang, H. Wang, H. Ding, Y. Lu, Z. Xu, Y. Zhou, B. Srinivasan, Q. Yan, Y. Chen, H. Ding, P. Xu, and L. L. Cheong, "A systematic survey of automatic prompt optimization techniques," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, preprint: arXiv:2502.16923. [Online]. Available: https://aclanthology.org/2025.emnlp-main.1681/

[13] M. Besta, T. Hoefler *et al.*, "Demystifying chains, trees, and graphs of thoughts," *arXiv preprint arXiv:2401.14295*, 2024. [Online]. Available: https://arxiv.org/abs/2401.14295

[14] Z. Li, B. Peng, P. He, M. Galley, J. Gao, and X. Yan, "Guiding large language models via directional stimulus prompting," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, preprint: arXiv:2302.11520. [Online]. Available: https://arxiv.org/pdf/2302.11520

[15] OWASP, "Owasp top 10 for large language model applications," OWASP Project, 2025, accessed: 2025-12-28. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[16] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," NIST, Tech. Rep. NIST AI 600-1, Jul. 2024. [Online]. Available: https://doi.org/10.6028/NIST.AI.600-1

[17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022, presented at ICLR 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[18] X. Liu, Z. Yu, Y. Zhang, N. Zhang, and C. Xiao, "Automatic and universal prompt injection attacks against large language models," *arXiv preprint arXiv:2403.04957*, 2024. [Online]. Available: https://arxiv.org/abs/2403.04957

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:39 UTC from IEEE Xplore. Restrictions apply.